

User Management (Admin & Customization)

16. How do you customize the Django admin to display extra fields of the User model?

We can customize the admin by creating a custom UserAdmin class. For example, if your user model has `phone_number`, you can add it to `list_display`:

```
from django.contrib import admin

from django.contrib.auth.admin import UserAdmin

from .models import CustomUser

class CustomUserAdmin(UserAdmin):

    list_display = ('email', 'first_name', 'last_name', 'phone_number', 'is_staff')

admin.site.register(CustomUser, CustomUserAdmin)
```

17. Write a query to fetch all active users who joined in the last 7 days.

```
from django.utils import timezone

from django.contrib.auth.models import User

from datetime import timedelta

seven_days_ago = timezone.now() - timedelta(days=7)

users = User.objects.filter(is_active=True, date_joined__gte=seven_days_ago)
```

18. What is the difference between `is_superuser`, `is_staff`, and `is_active` fields in the User model?

- `is_superuser` → full access to all permissions (can do anything).
- `is_staff` → can access the admin site but may not have all permissions.
- `is_active` → indicates if the user account is active; inactive users cannot log in.

19. How do you restrict certain views so that only **staff users** can access them?

We use the `@user_passes_test` decorator:

```
from django.contrib.auth.decorators import user_passes_test

@user_passes_test(lambda u: u.is_staff)

def staff_view(request):
```

20. What is the purpose of Django's **sessions framework** in authentication?

Django sessions store user-specific data between requests. It helps keep users logged in by storing a session ID in a cookie and user data in the backend, so you don't have to authenticate on every request.

Building a Persistence Data Layer

21. What is meant by **data persistence** in Django?

Data persistence means storing data in a database so that it remains even after the server stops or restarts. In Django, it ensures user info, posts, etc., are permanently saved.

22. Explain how Django ORM acts as a persistence layer.

The ORM (Object-Relational Mapping) lets you interact with the database using Python objects instead of raw SQL. Models represent tables, and ORM handles CRUD operations while keeping data consistent.

23. How do you configure Django to use PostgreSQL or MySQL instead of SQLite?

In settings.py, change the DATABASES setting:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql', # or 'mysql'  
        'NAME': 'mydb',  
        'USER': 'myuser',  
        'PASSWORD': 'mypassword',  
        'HOST': 'localhost',  
        'PORT': '5432',  
    }  
}
```

24. What is the role of **migrations** in ensuring data persistence?

Migrations track changes in models and apply them to the database. They ensure that your schema stays up-to-date without losing existing data.

25. How do you handle database connection pooling in Django?

Connection pooling can be configured via database driver settings. For example, PostgreSQL using psycopg2 supports pooling via `CONN_MAX_AGE` in `settings.py`:

```
DATABASES['default']['CONN_MAX_AGE'] = 600
```

Overview of Data Persistence Layers

26. What are the advantages of using Django ORM over writing raw SQL queries?

- Easier and faster to write queries in Python.
- Database-independent (can switch DB easily).
- Prevents SQL injection automatically.
- Provides relationships between models easily with foreign keys, one-to-one, many-to-many.

27. Explain the concept of **QuerySet caching** in Django ORM.

QuerySets are lazy and cached. When you first evaluate a QuerySet, the query is executed, and results are cached. Subsequent evaluations use the cache unless the QuerySet is modified.

28. How do you store session data in the database instead of cookies in Django?

In `settings.py`: `SESSION_ENGINE = 'django.contrib.sessions.backends.db'`

29. What is the difference between `select_related` and `prefetch_related` in Django ORM queries?

- `select_related` → **single SQL JOIN**, used for foreign keys or one-to-one relationships.
 - `prefetch_related` → **separate query and Python join**, used for many-to-many or reverse foreign key relations.
- This helps optimize database queries and reduce hits.

30. Write a query using ORM to fetch all users who have a specific permission assigned.

```
from django.contrib.auth.models import User, Permission

perm = Permission.objects.get(codename='change_article')

users = User.objects.filter(user_permissions=perm)
```